

imports

```
In [1]: import pandas as pd
import numpy as np
import tensorflow
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

dataset

```
In [2]: from pathlib import Path
root=Path('data')
root.mkdir(exist_ok=True)
path= root / "shakespeare.txt"
```

```
In [3]: with open(path, "r") as f:
    plays = f.read()
#plays
```

preprocessing

lower

```
In [4]: plays= plays.lower()
#plays
```

senntence list

```
In [5]: snentence=plays.split('\n')
#snentence
```

remove punctuation

```
In [6]: import string
exclude=string.punctuation + '\n' + '1'+ '2'+ '3'+ '4'+ '5'+ '6'+ '7'+ '8'+ '9'+ '0'
exclude
```

```
Out[6]: '!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~\n1234567890'
```

```
In [7]: def remove_punc(text):
    return text.translate(str.maketrans('', '', exclude))
```

```
In [8]: sentence_list=[]
for sen in range(len(snentence)):
    sentence_list.append(remove_punc(str(snentence[sen])))
len(sentence_list)
```

Out[8]: 124186

remove unwanted spaces

```
In [9]: for sen in range(len(sentence_list)):
        sentence_list[sen]=sentence_list[sen].strip(" ")
        sentence_list[:5]
```

```
Out[9]: ['from fairest creatures we desire increase',
        'that thereby beautys rose might never die',
        'but as the riper should by time decease',
        'his tender heir might bear his memory',
        'but thou contracted to thine own bright eyes']
```

remove null lists

```
In [10]: sentence_list = [s for s in sentence_list if s != ""]
```

```
In [11]: sentence_list[:5]
```

```
Out[11]: ['from fairest creatures we desire increase',
        'that thereby beautys rose might never die',
        'but as the riper should by time decease',
        'his tender heir might bear his memory',
        'but thou contracted to thine own bright eyes']
```

Add special tokens

```
In [12]: sentence_list = [f"[cls] {text} [sep]" for text in sentence_list]
```

```
In [13]: sentence_list[:5]
```

```
Out[13]: ['[cls] from fairest creatures we desire increase [sep]',
        '[cls] that thereby beautys rose might never die [sep]',
        '[cls] but as the riper should by time decease [sep]',
        '[cls] his tender heir might bear his memory [sep]',
        '[cls] but thou contracted to thine own bright eyes [sep]']
```

```
In [14]: len(sentence_list)
```

Out[14]: 114444

tokenization

```
In [15]: sen_tokenizer = Tokenizer(filters='0123456789!"#$$%&\'()*+,-./:;=?@\\^_`{|}~<>', oov
        sen_tokenizer.fit_on_texts(["[pad]" + sentence_list))
        sen_tokenizer.word_index["[pad]"]=0
        sen_tokenizer.index_word[0]="[pad]"
        sen_tokenizer
```

```
Out[15]: <keras.src.legacy.preprocessing.text.Tokenizer at 0x24215158440>
```

```
In [16]: sen_seq = sen_tokenizer.texts_to_sequences(sentence_list)
sen_seq[:5]
```

```
Out[16]: [[2, 52, 2032, 2033, 42, 478, 2698, 3],
          [2, 12, 2850, 2560, 1440, 220, 119, 231, 3],
          [2, 24, 26, 4, 10158, 91, 33, 102, 7047, 3],
          [2, 22, 772, 935, 220, 204, 22, 1556, 3],
          [2, 24, 28, 5002, 7, 240, 146, 1375, 159, 3]]
```

```
In [17]: # sequences to texts
sen_token = sen_tokenizer.sequences_to_texts(sen_seq)
sen_token[:5]
```

```
Out[17]: ['[cls] from fairest creatures we desire increase [sep]',
          '[cls] that thereby beautys rose might never die [sep]',
          '[cls] but as the riper should by time decease [sep]',
          '[cls] his tender heir might bear his memory [sep]',
          '[cls] but thou contracted to thine own bright eyes [sep]']
```

```
In [18]: len(sen_tokenizer.word_counts)
```

```
Out[18]: 27961
```

```
In [19]: sen_tokenizer.word_index['witch']
```

```
Out[19]: 1084
```

padding

```
In [20]: max_sen_len = max(len(seq) for seq in sen_seq)
```

```
In [21]: max_sen_len
```

```
Out[21]: 19
```

```
In [22]: seq_padded = pad_sequences(sen_seq, maxlen=max_sen_len, padding="post")
```

```
In [23]: seq_padded
```

```
Out[23]: array([[ 2,  52, 2032, ...,  0,  0,  0],
                 [ 2,  12, 2850, ...,  0,  0,  0],
                 [ 2,  24,  26, ...,  0,  0,  0],
                 ...,
                 [ 2,  59,  92, ...,  0,  0,  0],
                 [ 2,   5, 517, ...,  0,  0,  0],
                 [ 2,   4, 309, ...,  0,  0,  0]], dtype=int32)
```

```
In [24]: # sequences to texts
sen_token = sen_tokenizer.sequences_to_texts(seq_padded)
sen_token[:5]
```

```
Out[24]: ['[cls] from fairest creatures we desire increase [sep] [pad] [pad] [pad] [pad] [p
ad] [pad] [pad] [pad] [pad] [pad] [pad]',
 '[cls] that thereby beautys rose might never die [sep] [pad] [pad] [pad] [pad] [p
ad] [pad] [pad] [pad] [pad] [pad] [pad]',
 '[cls] but as the riper should by time decease [sep] [pad] [pad] [pad] [pad] [pa
d] [pad] [pad] [pad] [pad] [pad] [pad]',
 '[cls] his tender heir might bear his memory [sep] [pad] [pad] [pad] [pad] [pad]
[pad] [pad] [pad] [pad] [pad] [pad]',
 '[cls] but thou contracted to thine own bright eyes [sep] [pad] [pad] [pad] [pad]
[pad] [pad] [pad] [pad] [pad] [pad]']
```

```
In [25]: sen_tokenizer.word_index['<unk>']
```

```
Out[25]: 1
```

```
In [26]: sen_tokenizer.index_word[1]
```

```
Out[26]: '<unk>'
```

masked language modeling

```
In [27]: import torch
```

```
In [28]: seq_padded=torch.tensor(seq_padded)
```

```
In [29]: sen_tokenizer.word_index['[cls]']
```

```
Out[29]: 2
```

```
In [30]: sen_tokenizer.word_index['[sep]']
```

```
Out[30]: 3
```

```
In [31]: sen_tokenizer.word_index['[pad]']
```

```
Out[31]: 0
```

```
In [32]: sen_tokenizer.num_words = 27964
sen_tokenizer.word_counts=27964
vocab_size=sen_tokenizer.word_counts
vocab_size
```

```
Out[32]: 27964
```

```
In [33]: sen_tokenizer.index_word[27963]='[mask]'
sen_tokenizer.word_index['[mask]']=27963
```

```
In [34]: MASK_ID = sen_tokenizer.word_index['[mask]']
assert MASK_ID < vocab_size
```

```
In [35]: def simple_mask(
        input_ids,
        vocab_size,
        mask_token_id=27963,
        special_ids=[0,2,3],
        mlm_prob=0.25):

    input_ids = input_ids.clone()

    # Step 1: eligible positions
    rand = torch.rand_like(input_ids.float())
    maskable = ~torch.isin(input_ids, torch.tensor(special_ids, device=input_ids.device))
    mask = (rand < mlm_prob) & maskable

    # Step 2: Labels
    mlm_labels = input_ids.clone()
    mlm_labels[~mask] = -100 # ignore everywhere except masked

    # Step 3: inputs
    masked_x = input_ids.clone()
    masked_x[mask] = mask_token_id

    return masked_x.long(), mlm_labels.long()
```

```
In [36]: masked_x, mlm_labels = simple_mask(seq_padded, vocab_size=27964)
print("Masked inputs:\n", masked_x)
print("MLM labels:\n", mlm_labels)
```

Masked inputs:

```
tensor([[ 2, 52, 2032, ..., 0, 0, 0],
        [ 2, 12, 2850, ..., 0, 0, 0],
        [ 2, 24, 26, ..., 0, 0, 0],
        ...,
        [ 2, 59, 27963, ..., 0, 0, 0],
        [ 2, 5, 517, ..., 0, 0, 0],
        [ 2, 27963, 27963, ..., 0, 0, 0]])
```

MLM labels:

```
tensor([[ -100, -100, -100, ..., -100, -100, -100],
        [ -100, -100, -100, ..., -100, -100, -100],
        [ -100, -100, -100, ..., -100, -100, -100],
        ...,
        [ -100, -100, 92, ..., -100, -100, -100],
        [ -100, -100, -100, ..., -100, -100, -100],
        [ -100, 4, 309, ..., -100, -100, -100]])
```

```
In [37]: len(masked_x[1])
```

Out[37]: 19

```
In [38]: len(mlm_labels[1])
```

Out[38]: 19

```
In [39]: len(seq_padded)
```

Out[39]: 114444

```
In [40]: len(masked_x), len(mlm_labels)
```

Out[40]: (114444, 114444)

```
In [41]: sen_token = sen_tokenizer.sequences_to_texts(masked_x.numpy())
sen_token[:5]
```

Out[41]: ['[cls] from fairest [mask] [mask] desire increase [sep] [pad] [pad] [pad] [pad]
[pad] [pad] [pad] [pad] [pad] [pad] [pad]',
'[cls] that thereby beautys rose might never die [sep] [pad] [pad] [pad] [pad] [p
ad] [pad] [pad] [pad] [pad] [pad] [pad]',
'[cls] but as the [mask] should [mask] [mask] decease [sep] [pad] [pad] [pad] [pa
d] [pad] [pad] [pad] [pad] [pad] [pad]',
'[cls] his tender heir might bear [mask] memory [sep] [pad] [pad] [pad] [pad] [pa
d] [pad] [pad] [pad] [pad] [pad] [pad]',
'[cls] [mask] thou contracted to [mask] own bright eyes [sep] [pad] [pad] [pad]
[pad] [pad] [pad] [pad] [pad] [pad]']

```
In [42]: sen_token = sen_tokenizer.sequences_to_texts(mlm_labels.numpy())
sen_token[:5]
```

Out[42]: ['<unk> <unk> <unk> creatures we <unk> <unk> <unk> <unk> <unk> <unk> <unk> <
unk> <unk> <unk> <unk> <unk> <unk> <unk>',
'<unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk> <u
nk> <unk> <unk> <unk> <unk> <unk>',
'<unk> <unk> <unk> <unk> riper <unk> by time <unk> <unk> <unk> <unk> <unk> <unk>
<unk> <unk> <unk> <unk> <unk>',
'<unk> <unk> <unk> <unk> <unk> <unk> his <unk> <unk> <unk> <unk> <unk> <unk> <unk>
> <unk> <unk> <unk> <unk> <unk>',
'<unk> but <unk> <unk> <unk> thine <unk> <unk> <unk> <unk> <unk> <unk> <unk> <unk>
> <unk> <unk> <unk> <unk> <unk>']

```
In [43]: def decode_tokens(seq, index_word):
    return [
        index_word.get(tok.item(), '<unk>')
        for tok in seq
    ]

decoded = [decode_tokens(x, sen_tokenizer.index_word) for x in masked_x]
decoded[0] # will show '[mask]' instead of '<unk>'
```

```
Out[43]: ['[cls]',
          'from',
          'fairest',
          '[mask]',
          '[mask]',
          'desire',
          'increase',
          '[sep]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]',
          '[pad]']
```

```
In [ ]:
```

```
In [ ]:
```

Encoder Architecture

```
In [44]: import torch
import torch.nn as nn
import torch.nn.functional as F
import math
```

embeddings and positional encoding

```
In [45]: class embpos(nn.Module):
    def __init__(self, vocab_size, embdim, max_len=512):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embdim)

        # positional encoding (precomputed, CPU-safe)
        pe = torch.zeros(max_len, embdim)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, embdim, 2) * (-math.log(10000.0) / embdim)
        )
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)

        self.register_buffer("pe", pe.unsqueeze(0)) # (1, T, D)

    def forward(self, x):
```

```

B, T = x.size()
emb = self.embedding(x)                # (B, T, D)
emb = emb + self.pe[:, :T, :].to(emb.device)
return emb

```

```
In [46]: empo=embpos(27964,512)
```

```
In [47]: seq_padded[:6]
```

```

Out[47]: tensor([[ 2,  52, 2032, 2033,  42,  478, 2698,  3,  0,  0,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
                  [ 2,  12, 2850, 2560, 1440,  220,  119, 231,  3,  0,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
                  [ 2,  24,  26,  4, 10158,  91,  33, 102, 7047,  3,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
                  [ 2,  22,  772,  935,  220,  204,  22, 1556,  3,  0,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
                  [ 2,  24,  28, 5002,  7,  240, 146, 1375, 159,  3,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0],
                  [ 2, 10159,  34, 2699, 3112,  17, 15915, 7810,  3,  0,
                   0,  0,  0,  0,  0,  0,  0,  0,  0,  0]],
               dtype=torch.int32)

```

```
In [48]: import matplotlib.pyplot as plt
```

```

In [49]: def plot_pos(max_len = 50 ,embdim = 64 ,cmap='Spectral'):

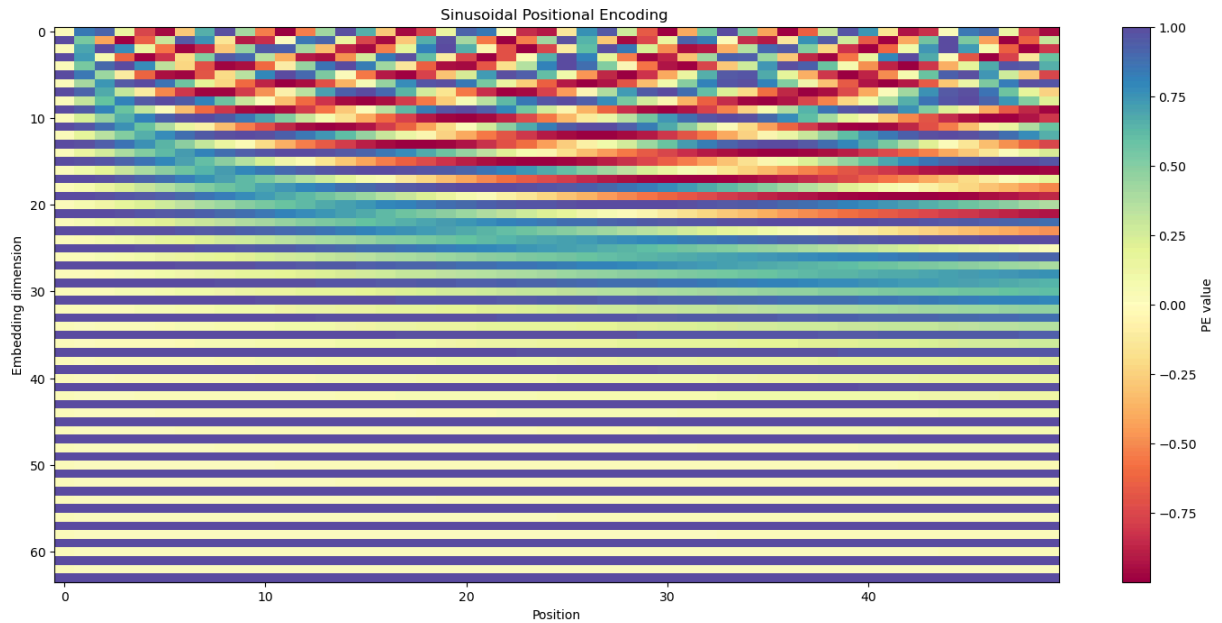
    # build PE from formula
    pe = torch.zeros(max_len, embdim)
    position = torch.arange(0, max_len).unsqueeze(1)
    div_term = torch.exp(
        torch.arange(0, embdim, 2) * (-math.log(10000.0) / embdim)
    )

    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)

    # plot
    plt.figure(figsize=(18, 8))
    plt.imshow(pe.T, aspect="auto", cmap=cmap)
    plt.colorbar(label="PE value")
    plt.xlabel("Position")
    plt.ylabel("Embedding dimension")
    plt.title("Sinusoidal Positional Encoding ")
    plt.show()

```

```
In [50]: plot_pos()
```



```
In [51]: res=empo(seq_padded[:6])
res[:2]
```

```
Out[51]: tensor([[[ 1.9966,  1.5137,  1.2352, ...,  1.7681, -1.3159,  0.8898],
 [ 0.5334,  0.3550,  0.3386, ...,  2.5502,  0.4111,  0.9933],
 [ 0.1708, -2.5510,  0.9770, ...,  0.0032,  0.5174,  1.3027],
 ...,
 [ 0.8127, -0.0797,  0.1099, ...,  1.0947, -0.8859, -0.4393],
 [ 0.1392,  0.6028, -0.7977, ...,  1.0947, -0.8858, -0.4393],
 [ 0.3496,  1.5383, -1.1565, ...,  1.0947, -0.8857, -0.4393]],

 [[ 1.9966,  1.5137,  1.2352, ...,  1.7681, -1.3159,  0.8898],
 [ 0.4415,  1.7227, -0.3525, ...,  0.7212,  0.1907,  2.2929],
 [ 1.0174, -0.7447,  1.7523, ...,  2.2704,  1.2563,  2.2945],
 ...,
 [ 0.8127, -0.0797,  0.1099, ...,  1.0947, -0.8859, -0.4393],
 [ 0.1392,  0.6028, -0.7977, ...,  1.0947, -0.8858, -0.4393],
 [ 0.3496,  1.5383, -1.1565, ...,  1.0947, -0.8857, -0.4393]]],
 grad_fn=<SliceBackward0>)
```

self-attention

```
In [52]: class SelfAttention(nn.Module):
def __init__(self, embdim):
    super().__init__()

    self.Wq = nn.Linear(embdim, embdim)
    self.Wk = nn.Linear(embdim, embdim)
    self.Wv = nn.Linear(embdim, embdim)

    self.scale = math.sqrt(embdim)

def forward(self, x):
    # x: (B, T, D)
```

[illegible]

```
nn.Linear(embdim * self.num_heads, embd
```

```
def forward(self, x):
    B, T, D = x.shape

    # split embedding into heads
    x = x.reshape(B, T, self.num_heads, self.head_dim)
    x = x.transpose(1, 2) # (B, H, T, D_head)

    head_outputs = []

    for i, head in enumerate(self.heads):
        head_x = x[:, i, :, :] # (B, T, D_head)
        head_outputs.append(head(head_x))

    concatenated = torch.cat(head_outputs, dim=-1)
    return self.output_linear(concatenated)
```

```
In [57]: multiat=MultiHeadAttention(512,8)
```

```
In [58]: answer=selfat(res)
```

```
In [59]: answer[:2]
```

```
Out[59]: tensor([[[ 0.4035,  0.6879, -0.3635, ...,  0.6847,  0.2497,  0.2739],
                   [ 0.2623,  0.9195, -0.9978, ...,  0.7326,  0.1229,  0.3237],
                   [ 0.2843,  0.8242, -0.6918, ...,  0.8209,  0.1977,  0.2978],
                   ...,
                   [ 0.2603,  0.8474, -0.5879, ...,  0.7028,  0.1255,  0.3245],
                   [ 0.2603,  0.8485, -0.5752, ...,  0.6979,  0.1343,  0.3280],
                   [ 0.2518,  0.8553, -0.5804, ...,  0.6933,  0.1436,  0.3275]],

                  [[ 0.5860,  0.3053, -0.4458, ...,  0.2720, -0.0114,  0.2182],
                   [ 0.4494,  0.4696, -0.5754, ...,  0.5119,  0.1259,  0.2227],
                   [ 0.4832,  0.4456, -0.5885, ...,  0.3778,  0.0178,  0.2504],
                   ...,
                   [ 0.3584,  0.6443, -0.8484, ...,  0.5012,  0.0924,  0.2097],
                   [ 0.3440,  0.6547, -0.8556, ...,  0.5200,  0.1038,  0.2166],
                   [ 0.3262,  0.6657, -0.8657, ...,  0.5331,  0.1099,  0.2265]]],
                grad_fn=<SliceBackward0>)
```

add residual connections and layer normalization

```
In [60]: class AddResidual_LayerNorm(nn.Module):
          def __init__(self, embdim):
              super().__init__()
              self.norm = nn.LayerNorm(embdim)

          def forward(self, x, sublayer):
              return x + sublayer(self.norm(x))
```

```
In [61]: adln=AddResidual_LayerNorm(512)
```

```
In [62]: anss=adln(answer,multiat)
```

```
In [63]: anss[:5]
```

```
Out[63]: tensor([[[[ 0.4398,  1.0485, -0.0974, ...,  1.2501, -0.0929, -0.0967],
 [ 0.2975,  1.2794, -0.7314, ...,  1.2992, -0.2202, -0.0469],
 [ 0.3200,  1.1847, -0.4254, ...,  1.3867, -0.1453, -0.0729],
 ...,
 [ 0.2959,  1.2079, -0.3215, ...,  1.2688, -0.2174, -0.0461],
 [ 0.2959,  1.2090, -0.3088, ...,  1.2639, -0.2086, -0.0427],
 [ 0.2873,  1.2158, -0.3140, ...,  1.2593, -0.1994, -0.0431]],

 [[ 0.5777,  0.7229, -0.1342, ...,  0.9102, -0.2856, -0.1109],
 [ 0.4412,  0.8871, -0.2639, ...,  1.1503, -0.1486, -0.1062],
 [ 0.4749,  0.8629, -0.2771, ...,  1.0161, -0.2565, -0.0784],
 ...,
 [ 0.3501,  1.0614, -0.5370, ...,  1.1397, -0.1821, -0.1191],
 [ 0.3357,  1.0718, -0.5442, ...,  1.1584, -0.1707, -0.1121],
 [ 0.3178,  1.0828, -0.5543, ...,  1.1716, -0.1647, -0.1023]],

 [[ 0.3393,  0.8754, -0.2618, ...,  1.1994,  0.0238, -0.3109],
 [ 0.2648,  0.8827, -0.2177, ...,  1.2655,  0.0554, -0.3114],
 [ 0.2823,  0.9903, -0.4873, ...,  1.3403, -0.0069, -0.3411],
 ...,
 [ 0.3148,  0.9412, -0.4289, ...,  1.2315, -0.0794, -0.2423],
 [ 0.3147,  0.9356, -0.4243, ...,  1.2251, -0.0820, -0.2514],
 [ 0.3155,  0.9383, -0.4353, ...,  1.2263, -0.0857, -0.2525]],

 [[ 0.4419,  1.0494, -0.1751, ...,  1.3329, -0.0445, -0.0463],
 [ 0.3359,  1.0726, -0.3475, ...,  1.2999, -0.1020,  0.0910],
 [ 0.2951,  1.1353, -0.4731, ...,  1.4119, -0.1438,  0.0432],
 ...,
 [ 0.3690,  1.1434, -0.4362, ...,  1.3063, -0.1626,  0.0063],
 [ 0.3641,  1.1495, -0.4461, ...,  1.3173, -0.1634,  0.0115],
 [ 0.3526,  1.1593, -0.4652, ...,  1.3329, -0.1706,  0.0198]],

 [[ 0.4810,  0.6998, -0.2142, ...,  1.1507,  0.1694, -0.2536],
 [ 0.5007,  0.7009, -0.2173, ...,  1.1789,  0.1552, -0.2369],
 [ 0.5074,  0.8473, -0.3003, ...,  1.3040,  0.2083, -0.2099],
 ...,
 [ 0.3724,  0.9131, -0.4130, ...,  1.1760, -0.0050, -0.1796],
 [ 0.3802,  0.9011, -0.3974, ...,  1.1732, -0.0034, -0.1782],
 [ 0.3834,  0.8993, -0.4044, ...,  1.1770,  0.0031, -0.1803]]],
 grad_fn=<SliceBackward0>)
```

feed-forward network

[illegible]

```
def forward(self, x):
    return self.output_linear(x)
```

```
In [65]: fefo=FeedForward(512,1024)
```

```
In [66]: answ=fefo(anss)
```

```
In [67]: answ[:2]
```

```
Out[67]: tensor([[[ 0.0958,  0.1480, -0.1998, ..., -0.1390, -0.2283, -0.0721],
 [ 0.1135,  0.2031, -0.1757, ..., -0.1106, -0.2285, -0.1562],
 [ 0.0833,  0.1848, -0.1966, ..., -0.1516, -0.2516, -0.1513],
 ...,
 [ 0.1129,  0.1921, -0.1889, ..., -0.1130, -0.2124, -0.1306],
 [ 0.1143,  0.1913, -0.1887, ..., -0.1137, -0.2103, -0.1286],
 [ 0.1183,  0.1924, -0.1878, ..., -0.1116, -0.2059, -0.1280]],

 [[ 0.0293,  0.0679, -0.2004, ..., -0.0825, -0.2701, -0.0804],
 [ 0.0352,  0.0749, -0.2014, ..., -0.1132, -0.2502, -0.0943],
 [ 0.0310,  0.0802, -0.1943, ..., -0.0984, -0.2594, -0.0889],
 ...,
 [ 0.0648,  0.1342, -0.1786, ..., -0.0867, -0.2386, -0.1089],
 [ 0.0642,  0.1328, -0.1805, ..., -0.0887, -0.2367, -0.1108],
 [ 0.0643,  0.1312, -0.1815, ..., -0.0902, -0.2347, -0.1132]]],
 grad_fn=<SliceBackward0>)
```

```
In [68]: ansfefo=adln(answ, fefo)
```

```
In [69]: ansfefo[:2]
```

```
Out[69]: tensor([[[ 0.1482,  0.1533, -0.3739, ..., -0.0162, -0.0677, -0.4668],
 [ 0.0851,  0.2653, -0.2223, ...,  0.1278, -0.1774, -0.6304],
 [ 0.0773,  0.1951, -0.2848, ...,  0.0225, -0.1185, -0.6032],
 ...,
 [ 0.1481,  0.2115, -0.3120, ...,  0.0788, -0.0819, -0.5772],
 [ 0.1524,  0.2100, -0.3161, ...,  0.0777, -0.0759, -0.5754],
 [ 0.1574,  0.2128, -0.3140, ...,  0.0840, -0.0704, -0.5764]],

 [[ 0.0340, -0.0523, -0.3212, ..., -0.1310, -0.2151, -0.5345],
 [ 0.0536, -0.0511, -0.2959, ..., -0.1084, -0.1657, -0.5366],
 [ 0.0287, -0.0421, -0.3041, ..., -0.0899, -0.1762, -0.5460],
 ...,
 [ 0.0188,  0.0206, -0.2824, ..., -0.0199, -0.1740, -0.5873],
 [ 0.0244,  0.0164, -0.2807, ..., -0.0211, -0.1752, -0.5872],
 [ 0.0318,  0.0115, -0.2779, ..., -0.0171, -0.1766, -0.5895]]],
 grad_fn=<SliceBackward0>)
```

EncoderBlock

```
In [70]: class EncoderBlock(nn.Module):
def __init__(self, embdim, num_heads, ff_hidden_dim):
    super().__init__()
```

```

self.mha = MultiHeadAttention(embdim, num_heads)
self.ff = FeedForward(embdim, ff_hidden_dim)
self.attn_norm = AddResidual_LayerNorm(embdim)
self.ff_norm = AddResidual_LayerNorm(embdim)

def forward(self, x):

    # multi-head attention with residual + norm
    x = self.attn_norm(x, self.mha)
    # feed-forward network with residual + norm
    x = self.ff_norm(x, self.ff)

    return x

```

```

In [71]: embdim=512
         num_heads=8
         ff_hidden_dim=1024
         vocab_size=27964

```

```

In [72]: embedposi=embpos(vocab_size,embdim)

```

```

In [73]: answerr=embedposi(seq_padded[:6])

```

```

In [74]: answerr[:2]

```

```

Out[74]: tensor([[[ 0.5642,  1.4974,  0.5065, ...,  0.9716, -0.2660,  0.4364],
                  [ 1.0837,  1.8065, -0.4357, ...,  1.6586,  0.2560,  2.1619],
                  [ 0.6810,  0.1647,  1.4597, ...,  0.6288, -1.5733,  2.7924],
                  ...,
                  [ 1.2364,  0.7685,  0.9757, ...,  2.1338,  0.7477,  1.0595],
                  [ 0.5629,  1.4510,  0.0682, ...,  2.1338,  0.7478,  1.0595],
                  [ 0.7733,  2.3865, -0.2906, ...,  2.1338,  0.7479,  1.0595]],

                 [[ 0.5642,  1.4974,  0.5065, ...,  0.9716, -0.2660,  0.4364],
                  [ 0.7595, -0.1753,  0.8605, ...,  0.7023,  1.6351,  2.1389],
                  [ 1.7440, -0.2931, -1.7118, ...,  1.0228, -0.5407,  1.3771],
                  ...,
                  [ 1.2364,  0.7685,  0.9757, ...,  2.1338,  0.7477,  1.0595],
                  [ 0.5629,  1.4510,  0.0682, ...,  2.1338,  0.7478,  1.0595],
                  [ 0.7733,  2.3865, -0.2906, ...,  2.1338,  0.7479,  1.0595]]],
               grad_fn=<SliceBackward0>))

```

```

In [75]: encoder=EncoderBlock(embdim, num_heads, ff_hidden_dim)

```

```

In [76]: resu=encoder(answerr)

```

```

In [77]: resu[:2]

```

```
Out[77]: tensor([[[ 0.4043,  2.1226,  0.2656, ...,  1.0458, -0.4877,  0.6071],
                  [ 0.8984,  2.3187, -0.8728, ...,  1.3244,  0.1545,  2.0991],
                  [ 0.7099,  0.5387,  1.0393, ...,  0.3727, -1.1905,  2.6920],
                  ...,
                  [ 1.4967,  1.3938,  0.8320, ...,  1.9602,  0.6859,  0.9540],
                  [ 0.8321,  2.0885, -0.0694, ...,  1.9248,  0.6824,  0.9990],
                  [ 1.0483,  3.0005, -0.4398, ...,  1.8875,  0.6598,  1.0281]],

                [[ 0.3368,  2.0176,  0.2838, ...,  1.0213, -0.5610,  0.4356],
                  [ 0.8848,  0.2219,  0.7461, ...,  0.2549,  1.6554,  2.3103],
                  [ 1.8738,  0.1077, -2.1441, ...,  0.6625, -0.7972,  1.1490],
                  ...,
                  [ 1.4216,  1.2682,  0.8606, ...,  1.9182,  0.6148,  0.7631],
                  [ 0.7602,  1.9659, -0.0480, ...,  1.8957,  0.6105,  0.8052],
                  [ 0.9773,  2.8828, -0.4199, ...,  1.8642,  0.5894,  0.8294]]],
              grad_fn=<SliceBackward0>)
```

BERT

```
In [78]: class BERT(nn.Module):
          def __init__(self, vocab_size, embdim=512, num_heads=8, ff_hidden_dim=2048, num
              super().__init__()
              # embedding + positional encoding
              self.emb = embpos(vocab_size, embdim)

              # stacked encoder blocks
              self.layers = nn.ModuleList([
                  EncoderBlock(embdim, num_heads, ff_hidden_dim) for _ in range(num_layer
              ])

          def forward(self, x):

              x = self.emb(x) # (B, T, D)

              for layer in self.layers:
                  x = layer(x)

              return x # (B, T, D)
```

```
In [79]: vocab_size=27964
```

```
In [80]: device = 'cuda' if torch.cuda.is_available() else 'cpu'
          #device='cpu'
```

```
In [81]: model=BERT(vocab_size=vocab_size).to(device)
```

```
In [82]: model
```

```

Out[82]: BERT(
  (emb): embpos(
    (embedding): Embedding(27964, 512)
  )
  (layers): ModuleList(
    (0-5): 6 x EncoderBlock(
      (mha): MultiHeadAttention(
        (heads): ModuleList(
          (0-7): 8 x SelfAttention(
            (Wq): Linear(in_features=64, out_features=64, bias=True)
            (Wk): Linear(in_features=64, out_features=64, bias=True)
            (Wv): Linear(in_features=64, out_features=64, bias=True)
          )
        )
      (output_linear): Linear(in_features=512, out_features=512, bias=True)
      (deep_output_linear): Sequential(
        (0): Linear(in_features=4096, out_features=8192, bias=True)
        (1): ReLU()
        (2): Linear(in_features=8192, out_features=4096, bias=True)
        (3): ReLU()
        (4): Linear(in_features=4096, out_features=512, bias=True)
      )
    )
    (ff): FeedForward(
      (output_linear): Sequential(
        (0): Linear(in_features=512, out_features=2048, bias=True)
        (1): ReLU()
        (2): Linear(in_features=2048, out_features=512, bias=True)
      )
    )
    (attn_norm): AddResidual_LayerNorm(
      (norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
    )
    (ff_norm): AddResidual_LayerNorm(
      (norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
    )
  )
)

```

```

In [83]: predd=model(torch.tensor(seq_padded[:6]).to(device))
predd[:2]

```

C:\Users\Lenovo\AppData\Local\Temp\ipykernel_22912\2481985315.py:1: UserWarning: To copy construct from a tensor, it is recommended to use sourceTensor.detach().clone() or sourceTensor.detach().clone().requires_grad_(True), rather than torch.tensor(sourceTensor).

```
predd=model(torch.tensor(seq_padded[:6]).to(device))
```

```
Out[83]: tensor([[-0.0628, -0.2044, -0.1805, ..., 2.3207, -0.8126, 1.5795],
                [-0.3610, 0.9056, 0.3332, ..., 3.5618, -1.6966, 0.9392],
                [ 0.5817, 2.2870, 2.2939, ..., 3.8194, -0.7845, 1.8868],
                ...,
                [-0.7970, -0.0404, -0.0977, ..., 2.1866, -0.2818, 2.4273],
                [-1.4278, 0.7093, -0.9745, ..., 2.0887, -0.2558, 2.3658],
                [-1.1986, 1.6733, -1.3885, ..., 1.9769, -0.2715, 2.3144]],

                [[-0.1265, -0.2174, -0.1710, ..., 2.2978, -0.6085, 1.3292],
                [ 0.2575, 1.6922, 1.6886, ..., 1.9375, -0.7723, -0.0101],
                [ 1.5458, 0.0506, 0.5641, ..., 2.8933, -0.9216, 2.6237],
                ...,
                [-0.9139, -0.0052, -0.0954, ..., 2.1553, -0.0122, 2.2087],
                [-1.5619, 0.7411, -0.9574, ..., 2.0675, 0.0476, 2.1596],
                [-1.3614, 1.6847, -1.3750, ..., 1.9740, 0.0394, 2.1232]]],
                device='cuda:0', grad_fn=<SliceBackward0>)
```

MLMHead

```
In [84]: class MLMHead(nn.Module):
        def __init__(self, embdim, vocab_size, hidden_dim=1024):
            super().__init__()
            self.mlp = nn.Sequential(
                nn.Linear(embdim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, hidden_dim),
                nn.ReLU(),
                nn.Linear(hidden_dim, vocab_size) # final logits for each token
            )

        def forward(self, x):
            # x: (B, T, embdim)
            return self.mlp(x) # (B, T, vocab_size)
```

```
In [85]: mlm_head=MLMHead(embdim, vocab_size).to(device)
```

```
In [86]: outtt=mlm_head(pred)
```

```
In [87]: outtt[:2]
```

```
Out[87]: tensor([[-0.0994,  0.0880,  0.0438, ..., -0.1985,  0.0407, -0.0155],
                [-0.0181,  0.0680, -0.0685, ..., -0.0546,  0.1696,  0.0298],
                [-0.1513,  0.0595,  0.0568, ..., -0.0087, -0.0628,  0.0066],
                ...,
                [-0.0667,  0.1986,  0.0527, ..., -0.1374, -0.0727, -0.1515],
                [-0.0414,  0.2011,  0.0602, ..., -0.1224, -0.0836, -0.1227],
                [-0.0348,  0.2234,  0.0407, ..., -0.1014, -0.0878, -0.1089]],

                [[-0.0886,  0.0849,  0.0462, ..., -0.1983,  0.0257, -0.0109],
                [-0.0615,  0.1220, -0.0710, ..., -0.0051,  0.0352, -0.0940],
                [-0.0153,  0.0794, -0.0577, ..., -0.0275, -0.0499, -0.2362],
                ...,
                [-0.0508,  0.2105,  0.0272, ..., -0.1492, -0.0798, -0.1447],
                [-0.0294,  0.2082,  0.0294, ..., -0.1353, -0.0888, -0.1155],
                [-0.0205,  0.2403,  0.0157, ..., -0.1098, -0.0953, -0.1005]]],
                device='cuda:0', grad_fn=<SliceBackward0>)
```

training

```
In [88]: from torch.utils.data import DataLoader
```

custom dataset

```
In [89]: class custom_dataset(torch.utils.data.Dataset):
        def __init__(self, sequences, vocab_size, mask_token_id=27963, mlm_prob=0.15, s
            self.sequences = sequences
            self.vocab_size = vocab_size
            self.mask_token_id = mask_token_id
            self.mlm_prob = mlm_prob
            self.special_ids = special_ids

        def __len__(self):
            return len(self.sequences)

        def __getitem__(self, idx):
            seq = self.sequences[idx]
            masked_x, mlm_labels = simple_mask(seq, self.vocab_size,
                                                mask_token_id=self.mask_token_id,
                                                mlm_prob=self.mlm_prob,
                                                special_ids=self.special_ids)

            return masked_x, mlm_labels
```

```
In [90]: dataset = custom_dataset(seq_padded, vocab_size=27964)
```

```
In [91]: dataloader=DataLoader(dataset,100)
```

```
In [92]: optimizer = torch.optim.Adam(model.parameters(), lr=1e-5)
        loss_fn = nn.CrossEntropyLoss(ignore_index=-100)
```

```
In [93]: print(masked_x.max(), masked_x.min())
        print(model.emb.embedding.num_embeddings)
```

```
tensor(27963) tensor(0)  
27964
```

```
In [94]: ### recording parameters for BERT
```

```
In [95]: for name, param in model.named_parameters():  
         print(name, param.requires_grad)
```

emb.embedding.weight True
layers.0.mha.heads.0.Wq.weight True
layers.0.mha.heads.0.Wq.bias True
layers.0.mha.heads.0.Wk.weight True
layers.0.mha.heads.0.Wk.bias True
layers.0.mha.heads.0.Wv.weight True
layers.0.mha.heads.0.Wv.bias True
layers.0.mha.heads.1.Wq.weight True
layers.0.mha.heads.1.Wq.bias True
layers.0.mha.heads.1.Wk.weight True
layers.0.mha.heads.1.Wk.bias True
layers.0.mha.heads.1.Wv.weight True
layers.0.mha.heads.1.Wv.bias True
layers.0.mha.heads.2.Wq.weight True
layers.0.mha.heads.2.Wq.bias True
layers.0.mha.heads.2.Wk.weight True
layers.0.mha.heads.2.Wk.bias True
layers.0.mha.heads.2.Wv.weight True
layers.0.mha.heads.2.Wv.bias True
layers.0.mha.heads.3.Wq.weight True
layers.0.mha.heads.3.Wq.bias True
layers.0.mha.heads.3.Wk.weight True
layers.0.mha.heads.3.Wk.bias True
layers.0.mha.heads.3.Wv.weight True
layers.0.mha.heads.3.Wv.bias True
layers.0.mha.heads.4.Wq.weight True
layers.0.mha.heads.4.Wq.bias True
layers.0.mha.heads.4.Wk.weight True
layers.0.mha.heads.4.Wk.bias True
layers.0.mha.heads.4.Wv.weight True
layers.0.mha.heads.4.Wv.bias True
layers.0.mha.heads.5.Wq.weight True
layers.0.mha.heads.5.Wq.bias True
layers.0.mha.heads.5.Wk.weight True
layers.0.mha.heads.5.Wk.bias True
layers.0.mha.heads.5.Wv.weight True
layers.0.mha.heads.5.Wv.bias True
layers.0.mha.heads.6.Wq.weight True
layers.0.mha.heads.6.Wq.bias True
layers.0.mha.heads.6.Wk.weight True
layers.0.mha.heads.6.Wk.bias True
layers.0.mha.heads.6.Wv.weight True
layers.0.mha.heads.6.Wv.bias True
layers.0.mha.heads.7.Wq.weight True
layers.0.mha.heads.7.Wq.bias True
layers.0.mha.heads.7.Wk.weight True
layers.0.mha.heads.7.Wk.bias True
layers.0.mha.heads.7.Wv.weight True
layers.0.mha.heads.7.Wv.bias True
layers.0.mha.output_linear.weight True
layers.0.mha.output_linear.bias True
layers.0.mha.deep_output_linear.0.weight True
layers.0.mha.deep_output_linear.0.bias True
layers.0.mha.deep_output_linear.2.weight True
layers.0.mha.deep_output_linear.2.bias True
layers.0.mha.deep_output_linear.4.weight True

layers.0.mha.deep_output_linear.4.bias True
layers.0.ff.output_linear.0.weight True
layers.0.ff.output_linear.0.bias True
layers.0.ff.output_linear.2.weight True
layers.0.ff.output_linear.2.bias True
layers.0.attn_norm.norm.weight True
layers.0.attn_norm.norm.bias True
layers.0.ff_norm.norm.weight True
layers.0.ff_norm.norm.bias True
layers.1.mha.heads.0.Wq.weight True
layers.1.mha.heads.0.Wq.bias True
layers.1.mha.heads.0.Wk.weight True
layers.1.mha.heads.0.Wk.bias True
layers.1.mha.heads.0.Wv.weight True
layers.1.mha.heads.0.Wv.bias True
layers.1.mha.heads.1.Wq.weight True
layers.1.mha.heads.1.Wq.bias True
layers.1.mha.heads.1.Wk.weight True
layers.1.mha.heads.1.Wk.bias True
layers.1.mha.heads.1.Wv.weight True
layers.1.mha.heads.1.Wv.bias True
layers.1.mha.heads.2.Wq.weight True
layers.1.mha.heads.2.Wq.bias True
layers.1.mha.heads.2.Wk.weight True
layers.1.mha.heads.2.Wk.bias True
layers.1.mha.heads.2.Wv.weight True
layers.1.mha.heads.2.Wv.bias True
layers.1.mha.heads.3.Wq.weight True
layers.1.mha.heads.3.Wq.bias True
layers.1.mha.heads.3.Wk.weight True
layers.1.mha.heads.3.Wk.bias True
layers.1.mha.heads.3.Wv.weight True
layers.1.mha.heads.3.Wv.bias True
layers.1.mha.heads.4.Wq.weight True
layers.1.mha.heads.4.Wq.bias True
layers.1.mha.heads.4.Wk.weight True
layers.1.mha.heads.4.Wk.bias True
layers.1.mha.heads.4.Wv.weight True
layers.1.mha.heads.4.Wv.bias True
layers.1.mha.heads.5.Wq.weight True
layers.1.mha.heads.5.Wq.bias True
layers.1.mha.heads.5.Wk.weight True
layers.1.mha.heads.5.Wk.bias True
layers.1.mha.heads.5.Wv.weight True
layers.1.mha.heads.5.Wv.bias True
layers.1.mha.heads.6.Wq.weight True
layers.1.mha.heads.6.Wq.bias True
layers.1.mha.heads.6.Wk.weight True
layers.1.mha.heads.6.Wk.bias True
layers.1.mha.heads.6.Wv.weight True
layers.1.mha.heads.6.Wv.bias True
layers.1.mha.heads.7.Wq.weight True
layers.1.mha.heads.7.Wq.bias True
layers.1.mha.heads.7.Wk.weight True
layers.1.mha.heads.7.Wk.bias True
layers.1.mha.heads.7.Wv.weight True

layers.1.mha.heads.7.Wv.bias True
layers.1.mha.output_linear.weight True
layers.1.mha.output_linear.bias True
layers.1.mha.deep_output_linear.0.weight True
layers.1.mha.deep_output_linear.0.bias True
layers.1.mha.deep_output_linear.2.weight True
layers.1.mha.deep_output_linear.2.bias True
layers.1.mha.deep_output_linear.4.weight True
layers.1.mha.deep_output_linear.4.bias True
layers.1.ff.output_linear.0.weight True
layers.1.ff.output_linear.0.bias True
layers.1.ff.output_linear.2.weight True
layers.1.ff.output_linear.2.bias True
layers.1.attn_norm.norm.weight True
layers.1.attn_norm.norm.bias True
layers.1.ff_norm.norm.weight True
layers.1.ff_norm.norm.bias True
layers.2.mha.heads.0.Wq.weight True
layers.2.mha.heads.0.Wq.bias True
layers.2.mha.heads.0.Wk.weight True
layers.2.mha.heads.0.Wk.bias True
layers.2.mha.heads.0.Wv.weight True
layers.2.mha.heads.0.Wv.bias True
layers.2.mha.heads.1.Wq.weight True
layers.2.mha.heads.1.Wq.bias True
layers.2.mha.heads.1.Wk.weight True
layers.2.mha.heads.1.Wk.bias True
layers.2.mha.heads.1.Wv.weight True
layers.2.mha.heads.1.Wv.bias True
layers.2.mha.heads.2.Wq.weight True
layers.2.mha.heads.2.Wq.bias True
layers.2.mha.heads.2.Wk.weight True
layers.2.mha.heads.2.Wk.bias True
layers.2.mha.heads.2.Wv.weight True
layers.2.mha.heads.2.Wv.bias True
layers.2.mha.heads.3.Wq.weight True
layers.2.mha.heads.3.Wq.bias True
layers.2.mha.heads.3.Wk.weight True
layers.2.mha.heads.3.Wk.bias True
layers.2.mha.heads.3.Wv.weight True
layers.2.mha.heads.3.Wv.bias True
layers.2.mha.heads.4.Wq.weight True
layers.2.mha.heads.4.Wq.bias True
layers.2.mha.heads.4.Wk.weight True
layers.2.mha.heads.4.Wk.bias True
layers.2.mha.heads.4.Wv.weight True
layers.2.mha.heads.4.Wv.bias True
layers.2.mha.heads.5.Wq.weight True
layers.2.mha.heads.5.Wq.bias True
layers.2.mha.heads.5.Wk.weight True
layers.2.mha.heads.5.Wk.bias True
layers.2.mha.heads.5.Wv.weight True
layers.2.mha.heads.5.Wv.bias True
layers.2.mha.heads.6.Wq.weight True
layers.2.mha.heads.6.Wq.bias True
layers.2.mha.heads.6.Wk.weight True

layers.2.mha.heads.6.Wk.bias True
layers.2.mha.heads.6.Wv.weight True
layers.2.mha.heads.6.Wv.bias True
layers.2.mha.heads.7.Wq.weight True
layers.2.mha.heads.7.Wq.bias True
layers.2.mha.heads.7.Wk.weight True
layers.2.mha.heads.7.Wk.bias True
layers.2.mha.heads.7.Wv.weight True
layers.2.mha.heads.7.Wv.bias True
layers.2.mha.output_linear.weight True
layers.2.mha.output_linear.bias True
layers.2.mha.deep_output_linear.0.weight True
layers.2.mha.deep_output_linear.0.bias True
layers.2.mha.deep_output_linear.2.weight True
layers.2.mha.deep_output_linear.2.bias True
layers.2.mha.deep_output_linear.4.weight True
layers.2.mha.deep_output_linear.4.bias True
layers.2.ff.output_linear.0.weight True
layers.2.ff.output_linear.0.bias True
layers.2.ff.output_linear.2.weight True
layers.2.ff.output_linear.2.bias True
layers.2.attn_norm.norm.weight True
layers.2.attn_norm.norm.bias True
layers.2.ff_norm.norm.weight True
layers.2.ff_norm.norm.bias True
layers.3.mha.heads.0.Wq.weight True
layers.3.mha.heads.0.Wq.bias True
layers.3.mha.heads.0.Wk.weight True
layers.3.mha.heads.0.Wk.bias True
layers.3.mha.heads.0.Wv.weight True
layers.3.mha.heads.0.Wv.bias True
layers.3.mha.heads.1.Wq.weight True
layers.3.mha.heads.1.Wq.bias True
layers.3.mha.heads.1.Wk.weight True
layers.3.mha.heads.1.Wk.bias True
layers.3.mha.heads.1.Wv.weight True
layers.3.mha.heads.1.Wv.bias True
layers.3.mha.heads.2.Wq.weight True
layers.3.mha.heads.2.Wq.bias True
layers.3.mha.heads.2.Wk.weight True
layers.3.mha.heads.2.Wk.bias True
layers.3.mha.heads.2.Wv.weight True
layers.3.mha.heads.2.Wv.bias True
layers.3.mha.heads.3.Wq.weight True
layers.3.mha.heads.3.Wq.bias True
layers.3.mha.heads.3.Wk.weight True
layers.3.mha.heads.3.Wk.bias True
layers.3.mha.heads.3.Wv.weight True
layers.3.mha.heads.3.Wv.bias True
layers.3.mha.heads.4.Wq.weight True
layers.3.mha.heads.4.Wq.bias True
layers.3.mha.heads.4.Wk.weight True
layers.3.mha.heads.4.Wk.bias True
layers.3.mha.heads.4.Wv.weight True
layers.3.mha.heads.4.Wv.bias True
layers.3.mha.heads.5.Wq.weight True

layers.3.mha.heads.5.Wq.bias True
layers.3.mha.heads.5.Wk.weight True
layers.3.mha.heads.5.Wk.bias True
layers.3.mha.heads.5.Wv.weight True
layers.3.mha.heads.5.Wv.bias True
layers.3.mha.heads.6.Wq.weight True
layers.3.mha.heads.6.Wq.bias True
layers.3.mha.heads.6.Wk.weight True
layers.3.mha.heads.6.Wk.bias True
layers.3.mha.heads.6.Wv.weight True
layers.3.mha.heads.6.Wv.bias True
layers.3.mha.heads.7.Wq.weight True
layers.3.mha.heads.7.Wq.bias True
layers.3.mha.heads.7.Wk.weight True
layers.3.mha.heads.7.Wk.bias True
layers.3.mha.heads.7.Wv.weight True
layers.3.mha.heads.7.Wv.bias True
layers.3.mha.output_linear.weight True
layers.3.mha.output_linear.bias True
layers.3.mha.deep_output_linear.0.weight True
layers.3.mha.deep_output_linear.0.bias True
layers.3.mha.deep_output_linear.2.weight True
layers.3.mha.deep_output_linear.2.bias True
layers.3.mha.deep_output_linear.4.weight True
layers.3.mha.deep_output_linear.4.bias True
layers.3.ff.output_linear.0.weight True
layers.3.ff.output_linear.0.bias True
layers.3.ff.output_linear.2.weight True
layers.3.ff.output_linear.2.bias True
layers.3.attn_norm.norm.weight True
layers.3.attn_norm.norm.bias True
layers.3.ff_norm.norm.weight True
layers.3.ff_norm.norm.bias True
layers.4.mha.heads.0.Wq.weight True
layers.4.mha.heads.0.Wq.bias True
layers.4.mha.heads.0.Wk.weight True
layers.4.mha.heads.0.Wk.bias True
layers.4.mha.heads.0.Wv.weight True
layers.4.mha.heads.0.Wv.bias True
layers.4.mha.heads.1.Wq.weight True
layers.4.mha.heads.1.Wq.bias True
layers.4.mha.heads.1.Wk.weight True
layers.4.mha.heads.1.Wk.bias True
layers.4.mha.heads.1.Wv.weight True
layers.4.mha.heads.1.Wv.bias True
layers.4.mha.heads.2.Wq.weight True
layers.4.mha.heads.2.Wq.bias True
layers.4.mha.heads.2.Wk.weight True
layers.4.mha.heads.2.Wk.bias True
layers.4.mha.heads.2.Wv.weight True
layers.4.mha.heads.2.Wv.bias True
layers.4.mha.heads.3.Wq.weight True
layers.4.mha.heads.3.Wq.bias True
layers.4.mha.heads.3.Wk.weight True
layers.4.mha.heads.3.Wk.bias True
layers.4.mha.heads.3.Wv.weight True

layers.4.mha.heads.3.Wv.bias True
layers.4.mha.heads.4.Wq.weight True
layers.4.mha.heads.4.Wq.bias True
layers.4.mha.heads.4.Wk.weight True
layers.4.mha.heads.4.Wk.bias True
layers.4.mha.heads.4.Wv.weight True
layers.4.mha.heads.4.Wv.bias True
layers.4.mha.heads.5.Wq.weight True
layers.4.mha.heads.5.Wq.bias True
layers.4.mha.heads.5.Wk.weight True
layers.4.mha.heads.5.Wk.bias True
layers.4.mha.heads.5.Wv.weight True
layers.4.mha.heads.5.Wv.bias True
layers.4.mha.heads.6.Wq.weight True
layers.4.mha.heads.6.Wq.bias True
layers.4.mha.heads.6.Wk.weight True
layers.4.mha.heads.6.Wk.bias True
layers.4.mha.heads.6.Wv.weight True
layers.4.mha.heads.6.Wv.bias True
layers.4.mha.heads.7.Wq.weight True
layers.4.mha.heads.7.Wq.bias True
layers.4.mha.heads.7.Wk.weight True
layers.4.mha.heads.7.Wk.bias True
layers.4.mha.heads.7.Wv.weight True
layers.4.mha.heads.7.Wv.bias True
layers.4.mha.output_linear.weight True
layers.4.mha.output_linear.bias True
layers.4.mha.deep_output_linear.0.weight True
layers.4.mha.deep_output_linear.0.bias True
layers.4.mha.deep_output_linear.2.weight True
layers.4.mha.deep_output_linear.2.bias True
layers.4.mha.deep_output_linear.4.weight True
layers.4.mha.deep_output_linear.4.bias True
layers.4.ff.output_linear.0.weight True
layers.4.ff.output_linear.0.bias True
layers.4.ff.output_linear.2.weight True
layers.4.ff.output_linear.2.bias True
layers.4.attn_norm.norm.weight True
layers.4.attn_norm.norm.bias True
layers.4.ff_norm.norm.weight True
layers.4.ff_norm.norm.bias True
layers.5.mha.heads.0.Wq.weight True
layers.5.mha.heads.0.Wq.bias True
layers.5.mha.heads.0.Wk.weight True
layers.5.mha.heads.0.Wk.bias True
layers.5.mha.heads.0.Wv.weight True
layers.5.mha.heads.0.Wv.bias True
layers.5.mha.heads.1.Wq.weight True
layers.5.mha.heads.1.Wq.bias True
layers.5.mha.heads.1.Wk.weight True
layers.5.mha.heads.1.Wk.bias True
layers.5.mha.heads.1.Wv.weight True
layers.5.mha.heads.1.Wv.bias True
layers.5.mha.heads.2.Wq.weight True
layers.5.mha.heads.2.Wq.bias True
layers.5.mha.heads.2.Wk.weight True

```

layers.5.mha.heads.2.Wk.bias True
layers.5.mha.heads.2.Wv.weight True
layers.5.mha.heads.2.Wv.bias True
layers.5.mha.heads.3.Wq.weight True
layers.5.mha.heads.3.Wq.bias True
layers.5.mha.heads.3.Wk.weight True
layers.5.mha.heads.3.Wk.bias True
layers.5.mha.heads.3.Wv.weight True
layers.5.mha.heads.3.Wv.bias True
layers.5.mha.heads.4.Wq.weight True
layers.5.mha.heads.4.Wq.bias True
layers.5.mha.heads.4.Wk.weight True
layers.5.mha.heads.4.Wk.bias True
layers.5.mha.heads.4.Wv.weight True
layers.5.mha.heads.4.Wv.bias True
layers.5.mha.heads.5.Wq.weight True
layers.5.mha.heads.5.Wq.bias True
layers.5.mha.heads.5.Wk.weight True
layers.5.mha.heads.5.Wk.bias True
layers.5.mha.heads.5.Wv.weight True
layers.5.mha.heads.5.Wv.bias True
layers.5.mha.heads.6.Wq.weight True
layers.5.mha.heads.6.Wq.bias True
layers.5.mha.heads.6.Wk.weight True
layers.5.mha.heads.6.Wk.bias True
layers.5.mha.heads.6.Wv.weight True
layers.5.mha.heads.6.Wv.bias True
layers.5.mha.heads.7.Wq.weight True
layers.5.mha.heads.7.Wq.bias True
layers.5.mha.heads.7.Wk.weight True
layers.5.mha.heads.7.Wk.bias True
layers.5.mha.heads.7.Wv.weight True
layers.5.mha.heads.7.Wv.bias True
layers.5.mha.output_linear.weight True
layers.5.mha.output_linear.bias True
layers.5.mha.deep_output_linear.0.weight True
layers.5.mha.deep_output_linear.0.bias True
layers.5.mha.deep_output_linear.2.weight True
layers.5.mha.deep_output_linear.2.bias True
layers.5.mha.deep_output_linear.4.weight True
layers.5.mha.deep_output_linear.4.bias True
layers.5.ff.output_linear.0.weight True
layers.5.ff.output_linear.0.bias True
layers.5.ff.output_linear.2.weight True
layers.5.ff.output_linear.2.bias True
layers.5.attn_norm.norm.weight True
layers.5.attn_norm.norm.bias True
layers.5.ff_norm.norm.weight True
layers.5.ff_norm.norm.bias True

```

```

In [96]: total_params = sum(p.numel() for p in model.parameters())
         print(total_params)

```

444416000

```
In [97]: trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(trainable_params)
```

444416000

```
In [98]: num_param_tensors = len(list(model.named_parameters()))
print(num_param_tensors)
```

385

```
In [99]: all_trainable = all(p.requires_grad for p in model.parameters())
print("All parameters trainable:", all_trainable)
```

All parameters trainable: True

creating model for implementation

```
In [100... class MY_BERT(nn.Module):
    def __init__(self, vocab_size, embdim=512):
        super().__init__()
        # bert
        self.BERT_ = BERT(vocab_size=vocab_size)

        # mlm-head
        self.MY_mlm_head=MLMHead(embdim, vocab_size)

    def forward(self, x):

        x = self.BERT_(x)
        x = self.MY_mlm_head(x)

        return x
```

```
In [101... my_model=MY_BERT(vocab_size).to(device)
my_model
```

```

Out[101... MY_BERT(
  (BERT_): BERT(
    (emb): embpos(
      (embedding): Embedding(27964, 512)
    )
    (layers): ModuleList(
      (0-5): 6 x EncoderBlock(
        (mha): MultiHeadAttention(
          (heads): ModuleList(
            (0-7): 8 x SelfAttention(
              (Wq): Linear(in_features=64, out_features=64, bias=True)
              (Wk): Linear(in_features=64, out_features=64, bias=True)
              (Wv): Linear(in_features=64, out_features=64, bias=True)
            )
          )
          (output_linear): Linear(in_features=512, out_features=512, bias=True)
          (deep_output_linear): Sequential(
            (0): Linear(in_features=4096, out_features=8192, bias=True)
            (1): ReLU()
            (2): Linear(in_features=8192, out_features=4096, bias=True)
            (3): ReLU()
            (4): Linear(in_features=4096, out_features=512, bias=True)
          )
        )
      )
      (ff): FeedForward(
        (output_linear): Sequential(
          (0): Linear(in_features=512, out_features=2048, bias=True)
          (1): ReLU()
          (2): Linear(in_features=2048, out_features=512, bias=True)
        )
      )
      (attn_norm): AddResidual_LayerNorm(
        (norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      )
      (ff_norm): AddResidual_LayerNorm(
        (norm): LayerNorm((512,), eps=1e-05, elementwise_affine=True)
      )
    )
  )
)
(MY_mlm_head): MLMHead(
  (mlp): Sequential(
    (0): Linear(in_features=512, out_features=1024, bias=True)
    (1): ReLU()
    (2): Linear(in_features=1024, out_features=1024, bias=True)
    (3): ReLU()
    (4): Linear(in_features=1024, out_features=27964, bias=True)
  )
)
)

```

recording parameters for MY-MODEL

```

In [102... for name, param in my_model.named_parameters():

```

```
print(name, param.requires_grad)
```

BERT_.emb.embedding.weight True
BERT_.layers.0.mha.heads.0.Wq.weight True
BERT_.layers.0.mha.heads.0.Wq.bias True
BERT_.layers.0.mha.heads.0.Wk.weight True
BERT_.layers.0.mha.heads.0.Wk.bias True
BERT_.layers.0.mha.heads.0.Wv.weight True
BERT_.layers.0.mha.heads.0.Wv.bias True
BERT_.layers.0.mha.heads.1.Wq.weight True
BERT_.layers.0.mha.heads.1.Wq.bias True
BERT_.layers.0.mha.heads.1.Wk.weight True
BERT_.layers.0.mha.heads.1.Wk.bias True
BERT_.layers.0.mha.heads.1.Wv.weight True
BERT_.layers.0.mha.heads.1.Wv.bias True
BERT_.layers.0.mha.heads.2.Wq.weight True
BERT_.layers.0.mha.heads.2.Wq.bias True
BERT_.layers.0.mha.heads.2.Wk.weight True
BERT_.layers.0.mha.heads.2.Wk.bias True
BERT_.layers.0.mha.heads.2.Wv.weight True
BERT_.layers.0.mha.heads.2.Wv.bias True
BERT_.layers.0.mha.heads.3.Wq.weight True
BERT_.layers.0.mha.heads.3.Wq.bias True
BERT_.layers.0.mha.heads.3.Wk.weight True
BERT_.layers.0.mha.heads.3.Wk.bias True
BERT_.layers.0.mha.heads.3.Wv.weight True
BERT_.layers.0.mha.heads.3.Wv.bias True
BERT_.layers.0.mha.heads.4.Wq.weight True
BERT_.layers.0.mha.heads.4.Wq.bias True
BERT_.layers.0.mha.heads.4.Wk.weight True
BERT_.layers.0.mha.heads.4.Wk.bias True
BERT_.layers.0.mha.heads.4.Wv.weight True
BERT_.layers.0.mha.heads.4.Wv.bias True
BERT_.layers.0.mha.heads.5.Wq.weight True
BERT_.layers.0.mha.heads.5.Wq.bias True
BERT_.layers.0.mha.heads.5.Wk.weight True
BERT_.layers.0.mha.heads.5.Wk.bias True
BERT_.layers.0.mha.heads.5.Wv.weight True
BERT_.layers.0.mha.heads.5.Wv.bias True
BERT_.layers.0.mha.heads.6.Wq.weight True
BERT_.layers.0.mha.heads.6.Wq.bias True
BERT_.layers.0.mha.heads.6.Wk.weight True
BERT_.layers.0.mha.heads.6.Wk.bias True
BERT_.layers.0.mha.heads.6.Wv.weight True
BERT_.layers.0.mha.heads.6.Wv.bias True
BERT_.layers.0.mha.heads.7.Wq.weight True
BERT_.layers.0.mha.heads.7.Wq.bias True
BERT_.layers.0.mha.heads.7.Wk.weight True
BERT_.layers.0.mha.heads.7.Wk.bias True
BERT_.layers.0.mha.heads.7.Wv.weight True
BERT_.layers.0.mha.heads.7.Wv.bias True
BERT_.layers.0.mha.output_linear.weight True
BERT_.layers.0.mha.output_linear.bias True
BERT_.layers.0.mha.deep_output_linear.0.weight True
BERT_.layers.0.mha.deep_output_linear.0.bias True
BERT_.layers.0.mha.deep_output_linear.2.weight True
BERT_.layers.0.mha.deep_output_linear.2.bias True
BERT_.layers.0.mha.deep_output_linear.4.weight True

BERT_.layers.0.mha.deep_output_linear.4.bias True
BERT_.layers.0.ff.output_linear.0.weight True
BERT_.layers.0.ff.output_linear.0.bias True
BERT_.layers.0.ff.output_linear.2.weight True
BERT_.layers.0.ff.output_linear.2.bias True
BERT_.layers.0.attn_norm.norm.weight True
BERT_.layers.0.attn_norm.norm.bias True
BERT_.layers.0.ff_norm.norm.weight True
BERT_.layers.0.ff_norm.norm.bias True
BERT_.layers.1.mha.heads.0.Wq.weight True
BERT_.layers.1.mha.heads.0.Wq.bias True
BERT_.layers.1.mha.heads.0.Wk.weight True
BERT_.layers.1.mha.heads.0.Wk.bias True
BERT_.layers.1.mha.heads.0.Wv.weight True
BERT_.layers.1.mha.heads.0.Wv.bias True
BERT_.layers.1.mha.heads.1.Wq.weight True
BERT_.layers.1.mha.heads.1.Wq.bias True
BERT_.layers.1.mha.heads.1.Wk.weight True
BERT_.layers.1.mha.heads.1.Wk.bias True
BERT_.layers.1.mha.heads.1.Wv.weight True
BERT_.layers.1.mha.heads.1.Wv.bias True
BERT_.layers.1.mha.heads.2.Wq.weight True
BERT_.layers.1.mha.heads.2.Wq.bias True
BERT_.layers.1.mha.heads.2.Wk.weight True
BERT_.layers.1.mha.heads.2.Wk.bias True
BERT_.layers.1.mha.heads.2.Wv.weight True
BERT_.layers.1.mha.heads.2.Wv.bias True
BERT_.layers.1.mha.heads.3.Wq.weight True
BERT_.layers.1.mha.heads.3.Wq.bias True
BERT_.layers.1.mha.heads.3.Wk.weight True
BERT_.layers.1.mha.heads.3.Wk.bias True
BERT_.layers.1.mha.heads.3.Wv.weight True
BERT_.layers.1.mha.heads.3.Wv.bias True
BERT_.layers.1.mha.heads.4.Wq.weight True
BERT_.layers.1.mha.heads.4.Wq.bias True
BERT_.layers.1.mha.heads.4.Wk.weight True
BERT_.layers.1.mha.heads.4.Wk.bias True
BERT_.layers.1.mha.heads.4.Wv.weight True
BERT_.layers.1.mha.heads.4.Wv.bias True
BERT_.layers.1.mha.heads.5.Wq.weight True
BERT_.layers.1.mha.heads.5.Wq.bias True
BERT_.layers.1.mha.heads.5.Wk.weight True
BERT_.layers.1.mha.heads.5.Wk.bias True
BERT_.layers.1.mha.heads.5.Wv.weight True
BERT_.layers.1.mha.heads.5.Wv.bias True
BERT_.layers.1.mha.heads.6.Wq.weight True
BERT_.layers.1.mha.heads.6.Wq.bias True
BERT_.layers.1.mha.heads.6.Wk.weight True
BERT_.layers.1.mha.heads.6.Wk.bias True
BERT_.layers.1.mha.heads.6.Wv.weight True
BERT_.layers.1.mha.heads.6.Wv.bias True
BERT_.layers.1.mha.heads.7.Wq.weight True
BERT_.layers.1.mha.heads.7.Wq.bias True
BERT_.layers.1.mha.heads.7.Wk.weight True
BERT_.layers.1.mha.heads.7.Wk.bias True
BERT_.layers.1.mha.heads.7.Wv.weight True

BERT_.layers.1.mha.heads.7.Wv.bias True
BERT_.layers.1.mha.output_linear.weight True
BERT_.layers.1.mha.output_linear.bias True
BERT_.layers.1.mha.deep_output_linear.0.weight True
BERT_.layers.1.mha.deep_output_linear.0.bias True
BERT_.layers.1.mha.deep_output_linear.2.weight True
BERT_.layers.1.mha.deep_output_linear.2.bias True
BERT_.layers.1.mha.deep_output_linear.4.weight True
BERT_.layers.1.mha.deep_output_linear.4.bias True
BERT_.layers.1.ff.output_linear.0.weight True
BERT_.layers.1.ff.output_linear.0.bias True
BERT_.layers.1.ff.output_linear.2.weight True
BERT_.layers.1.ff.output_linear.2.bias True
BERT_.layers.1.attn_norm.norm.weight True
BERT_.layers.1.attn_norm.norm.bias True
BERT_.layers.1.ff_norm.norm.weight True
BERT_.layers.1.ff_norm.norm.bias True
BERT_.layers.2.mha.heads.0.Wq.weight True
BERT_.layers.2.mha.heads.0.Wq.bias True
BERT_.layers.2.mha.heads.0.Wk.weight True
BERT_.layers.2.mha.heads.0.Wk.bias True
BERT_.layers.2.mha.heads.0.Wv.weight True
BERT_.layers.2.mha.heads.0.Wv.bias True
BERT_.layers.2.mha.heads.1.Wq.weight True
BERT_.layers.2.mha.heads.1.Wq.bias True
BERT_.layers.2.mha.heads.1.Wk.weight True
BERT_.layers.2.mha.heads.1.Wk.bias True
BERT_.layers.2.mha.heads.1.Wv.weight True
BERT_.layers.2.mha.heads.1.Wv.bias True
BERT_.layers.2.mha.heads.2.Wq.weight True
BERT_.layers.2.mha.heads.2.Wq.bias True
BERT_.layers.2.mha.heads.2.Wk.weight True
BERT_.layers.2.mha.heads.2.Wk.bias True
BERT_.layers.2.mha.heads.2.Wv.weight True
BERT_.layers.2.mha.heads.2.Wv.bias True
BERT_.layers.2.mha.heads.3.Wq.weight True
BERT_.layers.2.mha.heads.3.Wq.bias True
BERT_.layers.2.mha.heads.3.Wk.weight True
BERT_.layers.2.mha.heads.3.Wk.bias True
BERT_.layers.2.mha.heads.3.Wv.weight True
BERT_.layers.2.mha.heads.3.Wv.bias True
BERT_.layers.2.mha.heads.4.Wq.weight True
BERT_.layers.2.mha.heads.4.Wq.bias True
BERT_.layers.2.mha.heads.4.Wk.weight True
BERT_.layers.2.mha.heads.4.Wk.bias True
BERT_.layers.2.mha.heads.4.Wv.weight True
BERT_.layers.2.mha.heads.4.Wv.bias True
BERT_.layers.2.mha.heads.5.Wq.weight True
BERT_.layers.2.mha.heads.5.Wq.bias True
BERT_.layers.2.mha.heads.5.Wk.weight True
BERT_.layers.2.mha.heads.5.Wk.bias True
BERT_.layers.2.mha.heads.5.Wv.weight True
BERT_.layers.2.mha.heads.5.Wv.bias True
BERT_.layers.2.mha.heads.6.Wq.weight True
BERT_.layers.2.mha.heads.6.Wq.bias True
BERT_.layers.2.mha.heads.6.Wk.weight True

BERT_.layers.2.mha.heads.6.Wk.bias True
BERT_.layers.2.mha.heads.6.Wv.weight True
BERT_.layers.2.mha.heads.6.Wv.bias True
BERT_.layers.2.mha.heads.7.Wq.weight True
BERT_.layers.2.mha.heads.7.Wq.bias True
BERT_.layers.2.mha.heads.7.Wk.weight True
BERT_.layers.2.mha.heads.7.Wk.bias True
BERT_.layers.2.mha.heads.7.Wv.weight True
BERT_.layers.2.mha.heads.7.Wv.bias True
BERT_.layers.2.mha.output_linear.weight True
BERT_.layers.2.mha.output_linear.bias True
BERT_.layers.2.mha.deep_output_linear.0.weight True
BERT_.layers.2.mha.deep_output_linear.0.bias True
BERT_.layers.2.mha.deep_output_linear.2.weight True
BERT_.layers.2.mha.deep_output_linear.2.bias True
BERT_.layers.2.mha.deep_output_linear.4.weight True
BERT_.layers.2.mha.deep_output_linear.4.bias True
BERT_.layers.2.ff.output_linear.0.weight True
BERT_.layers.2.ff.output_linear.0.bias True
BERT_.layers.2.ff.output_linear.2.weight True
BERT_.layers.2.ff.output_linear.2.bias True
BERT_.layers.2.attn_norm.norm.weight True
BERT_.layers.2.attn_norm.norm.bias True
BERT_.layers.2.ff_norm.norm.weight True
BERT_.layers.2.ff_norm.norm.bias True
BERT_.layers.3.mha.heads.0.Wq.weight True
BERT_.layers.3.mha.heads.0.Wq.bias True
BERT_.layers.3.mha.heads.0.Wk.weight True
BERT_.layers.3.mha.heads.0.Wk.bias True
BERT_.layers.3.mha.heads.0.Wv.weight True
BERT_.layers.3.mha.heads.0.Wv.bias True
BERT_.layers.3.mha.heads.1.Wq.weight True
BERT_.layers.3.mha.heads.1.Wq.bias True
BERT_.layers.3.mha.heads.1.Wk.weight True
BERT_.layers.3.mha.heads.1.Wk.bias True
BERT_.layers.3.mha.heads.1.Wv.weight True
BERT_.layers.3.mha.heads.1.Wv.bias True
BERT_.layers.3.mha.heads.2.Wq.weight True
BERT_.layers.3.mha.heads.2.Wq.bias True
BERT_.layers.3.mha.heads.2.Wk.weight True
BERT_.layers.3.mha.heads.2.Wk.bias True
BERT_.layers.3.mha.heads.2.Wv.weight True
BERT_.layers.3.mha.heads.2.Wv.bias True
BERT_.layers.3.mha.heads.3.Wq.weight True
BERT_.layers.3.mha.heads.3.Wq.bias True
BERT_.layers.3.mha.heads.3.Wk.weight True
BERT_.layers.3.mha.heads.3.Wk.bias True
BERT_.layers.3.mha.heads.3.Wv.weight True
BERT_.layers.3.mha.heads.3.Wv.bias True
BERT_.layers.3.mha.heads.4.Wq.weight True
BERT_.layers.3.mha.heads.4.Wq.bias True
BERT_.layers.3.mha.heads.4.Wk.weight True
BERT_.layers.3.mha.heads.4.Wk.bias True
BERT_.layers.3.mha.heads.4.Wv.weight True
BERT_.layers.3.mha.heads.4.Wv.bias True
BERT_.layers.3.mha.heads.5.Wq.weight True

BERT_.layers.3.mha.heads.5.Wq.bias True
BERT_.layers.3.mha.heads.5.Wk.weight True
BERT_.layers.3.mha.heads.5.Wk.bias True
BERT_.layers.3.mha.heads.5.Wv.weight True
BERT_.layers.3.mha.heads.5.Wv.bias True
BERT_.layers.3.mha.heads.6.Wq.weight True
BERT_.layers.3.mha.heads.6.Wq.bias True
BERT_.layers.3.mha.heads.6.Wk.weight True
BERT_.layers.3.mha.heads.6.Wk.bias True
BERT_.layers.3.mha.heads.6.Wv.weight True
BERT_.layers.3.mha.heads.6.Wv.bias True
BERT_.layers.3.mha.heads.7.Wq.weight True
BERT_.layers.3.mha.heads.7.Wq.bias True
BERT_.layers.3.mha.heads.7.Wk.weight True
BERT_.layers.3.mha.heads.7.Wk.bias True
BERT_.layers.3.mha.heads.7.Wv.weight True
BERT_.layers.3.mha.heads.7.Wv.bias True
BERT_.layers.3.mha.output_linear.weight True
BERT_.layers.3.mha.output_linear.bias True
BERT_.layers.3.mha.deep_output_linear.0.weight True
BERT_.layers.3.mha.deep_output_linear.0.bias True
BERT_.layers.3.mha.deep_output_linear.2.weight True
BERT_.layers.3.mha.deep_output_linear.2.bias True
BERT_.layers.3.mha.deep_output_linear.4.weight True
BERT_.layers.3.mha.deep_output_linear.4.bias True
BERT_.layers.3.ff.output_linear.0.weight True
BERT_.layers.3.ff.output_linear.0.bias True
BERT_.layers.3.ff.output_linear.2.weight True
BERT_.layers.3.ff.output_linear.2.bias True
BERT_.layers.3.attn_norm.norm.weight True
BERT_.layers.3.attn_norm.norm.bias True
BERT_.layers.3.ff_norm.norm.weight True
BERT_.layers.3.ff_norm.norm.bias True
BERT_.layers.4.mha.heads.0.Wq.weight True
BERT_.layers.4.mha.heads.0.Wq.bias True
BERT_.layers.4.mha.heads.0.Wk.weight True
BERT_.layers.4.mha.heads.0.Wk.bias True
BERT_.layers.4.mha.heads.0.Wv.weight True
BERT_.layers.4.mha.heads.0.Wv.bias True
BERT_.layers.4.mha.heads.1.Wq.weight True
BERT_.layers.4.mha.heads.1.Wq.bias True
BERT_.layers.4.mha.heads.1.Wk.weight True
BERT_.layers.4.mha.heads.1.Wk.bias True
BERT_.layers.4.mha.heads.1.Wv.weight True
BERT_.layers.4.mha.heads.1.Wv.bias True
BERT_.layers.4.mha.heads.2.Wq.weight True
BERT_.layers.4.mha.heads.2.Wq.bias True
BERT_.layers.4.mha.heads.2.Wk.weight True
BERT_.layers.4.mha.heads.2.Wk.bias True
BERT_.layers.4.mha.heads.2.Wv.weight True
BERT_.layers.4.mha.heads.2.Wv.bias True
BERT_.layers.4.mha.heads.3.Wq.weight True
BERT_.layers.4.mha.heads.3.Wq.bias True
BERT_.layers.4.mha.heads.3.Wk.weight True
BERT_.layers.4.mha.heads.3.Wk.bias True
BERT_.layers.4.mha.heads.3.Wv.weight True

BERT_.layers.4.mha.heads.3.Wv.bias True
BERT_.layers.4.mha.heads.4.Wq.weight True
BERT_.layers.4.mha.heads.4.Wq.bias True
BERT_.layers.4.mha.heads.4.Wk.weight True
BERT_.layers.4.mha.heads.4.Wk.bias True
BERT_.layers.4.mha.heads.4.Wv.weight True
BERT_.layers.4.mha.heads.4.Wv.bias True
BERT_.layers.4.mha.heads.5.Wq.weight True
BERT_.layers.4.mha.heads.5.Wq.bias True
BERT_.layers.4.mha.heads.5.Wk.weight True
BERT_.layers.4.mha.heads.5.Wk.bias True
BERT_.layers.4.mha.heads.5.Wv.weight True
BERT_.layers.4.mha.heads.5.Wv.bias True
BERT_.layers.4.mha.heads.6.Wq.weight True
BERT_.layers.4.mha.heads.6.Wq.bias True
BERT_.layers.4.mha.heads.6.Wk.weight True
BERT_.layers.4.mha.heads.6.Wk.bias True
BERT_.layers.4.mha.heads.6.Wv.weight True
BERT_.layers.4.mha.heads.6.Wv.bias True
BERT_.layers.4.mha.heads.7.Wq.weight True
BERT_.layers.4.mha.heads.7.Wq.bias True
BERT_.layers.4.mha.heads.7.Wk.weight True
BERT_.layers.4.mha.heads.7.Wk.bias True
BERT_.layers.4.mha.heads.7.Wv.weight True
BERT_.layers.4.mha.heads.7.Wv.bias True
BERT_.layers.4.mha.output_linear.weight True
BERT_.layers.4.mha.output_linear.bias True
BERT_.layers.4.mha.deep_output_linear.0.weight True
BERT_.layers.4.mha.deep_output_linear.0.bias True
BERT_.layers.4.mha.deep_output_linear.2.weight True
BERT_.layers.4.mha.deep_output_linear.2.bias True
BERT_.layers.4.mha.deep_output_linear.4.weight True
BERT_.layers.4.mha.deep_output_linear.4.bias True
BERT_.layers.4.ff.output_linear.0.weight True
BERT_.layers.4.ff.output_linear.0.bias True
BERT_.layers.4.ff.output_linear.2.weight True
BERT_.layers.4.ff.output_linear.2.bias True
BERT_.layers.4.attn_norm.norm.weight True
BERT_.layers.4.attn_norm.norm.bias True
BERT_.layers.4.ff_norm.norm.weight True
BERT_.layers.4.ff_norm.norm.bias True
BERT_.layers.5.mha.heads.0.Wq.weight True
BERT_.layers.5.mha.heads.0.Wq.bias True
BERT_.layers.5.mha.heads.0.Wk.weight True
BERT_.layers.5.mha.heads.0.Wk.bias True
BERT_.layers.5.mha.heads.0.Wv.weight True
BERT_.layers.5.mha.heads.0.Wv.bias True
BERT_.layers.5.mha.heads.1.Wq.weight True
BERT_.layers.5.mha.heads.1.Wq.bias True
BERT_.layers.5.mha.heads.1.Wk.weight True
BERT_.layers.5.mha.heads.1.Wk.bias True
BERT_.layers.5.mha.heads.1.Wv.weight True
BERT_.layers.5.mha.heads.1.Wv.bias True
BERT_.layers.5.mha.heads.2.Wq.weight True
BERT_.layers.5.mha.heads.2.Wq.bias True
BERT_.layers.5.mha.heads.2.Wk.weight True

BERT_.layers.5.mha.heads.2.Wk.bias True
BERT_.layers.5.mha.heads.2.Wv.weight True
BERT_.layers.5.mha.heads.2.Wv.bias True
BERT_.layers.5.mha.heads.3.Wq.weight True
BERT_.layers.5.mha.heads.3.Wq.bias True
BERT_.layers.5.mha.heads.3.Wk.weight True
BERT_.layers.5.mha.heads.3.Wk.bias True
BERT_.layers.5.mha.heads.3.Wv.weight True
BERT_.layers.5.mha.heads.3.Wv.bias True
BERT_.layers.5.mha.heads.4.Wq.weight True
BERT_.layers.5.mha.heads.4.Wq.bias True
BERT_.layers.5.mha.heads.4.Wk.weight True
BERT_.layers.5.mha.heads.4.Wk.bias True
BERT_.layers.5.mha.heads.4.Wv.weight True
BERT_.layers.5.mha.heads.4.Wv.bias True
BERT_.layers.5.mha.heads.5.Wq.weight True
BERT_.layers.5.mha.heads.5.Wq.bias True
BERT_.layers.5.mha.heads.5.Wk.weight True
BERT_.layers.5.mha.heads.5.Wk.bias True
BERT_.layers.5.mha.heads.5.Wv.weight True
BERT_.layers.5.mha.heads.5.Wv.bias True
BERT_.layers.5.mha.heads.6.Wq.weight True
BERT_.layers.5.mha.heads.6.Wq.bias True
BERT_.layers.5.mha.heads.6.Wk.weight True
BERT_.layers.5.mha.heads.6.Wk.bias True
BERT_.layers.5.mha.heads.6.Wv.weight True
BERT_.layers.5.mha.heads.6.Wv.bias True
BERT_.layers.5.mha.heads.7.Wq.weight True
BERT_.layers.5.mha.heads.7.Wq.bias True
BERT_.layers.5.mha.heads.7.Wk.weight True
BERT_.layers.5.mha.heads.7.Wk.bias True
BERT_.layers.5.mha.heads.7.Wv.weight True
BERT_.layers.5.mha.heads.7.Wv.bias True
BERT_.layers.5.mha.output_linear.weight True
BERT_.layers.5.mha.output_linear.bias True
BERT_.layers.5.mha.deep_output_linear.0.weight True
BERT_.layers.5.mha.deep_output_linear.0.bias True
BERT_.layers.5.mha.deep_output_linear.2.weight True
BERT_.layers.5.mha.deep_output_linear.2.bias True
BERT_.layers.5.mha.deep_output_linear.4.weight True
BERT_.layers.5.mha.deep_output_linear.4.bias True
BERT_.layers.5.ff.output_linear.0.weight True
BERT_.layers.5.ff.output_linear.0.bias True
BERT_.layers.5.ff.output_linear.2.weight True
BERT_.layers.5.ff.output_linear.2.bias True
BERT_.layers.5.attn_norm.norm.weight True
BERT_.layers.5.attn_norm.norm.bias True
BERT_.layers.5.ff_norm.norm.weight True
BERT_.layers.5.ff_norm.norm.bias True
MY_mlm_head.mlp.0.weight True
MY_mlm_head.mlp.0.bias True
MY_mlm_head.mlp.2.weight True
MY_mlm_head.mlp.2.bias True
MY_mlm_head.mlp.4.weight True
MY_mlm_head.mlp.4.bias True

```
In [103... total_params = sum(p.numel() for p in my_model.parameters())
print(total_params)
```

474654012

```
In [104... trainable_params = sum(p.numel() for p in my_model.parameters() if p.requires_grad)
print(trainable_params)
```

474654012

```
In [105... num_param_tensors = len(list(my_model.named_parameters()))
print(num_param_tensors)
```

391

```
In [106... all_trainable = all(p.requires_grad for p in my_model.parameters())
print("All parameters trainable:", all_trainable)
```

All parameters trainable: True

training for Bert only

```
In [107... def mlm_accuracy(logits, labels):

    preds = logits.argmax(dim=-1) # (B, T)

    mask = labels != -100

    if mask.sum() == 0:
        return 0.0

    correct = (preds[mask] == labels[mask]).sum().item()
    total = mask.sum().item()

    return correct / total
```

```
In [108... def train(EPOCH, batch_size):
    model.train()

    for epoch in range(EPOCH):
        total_loss = 0.0
        total_acc = 0.0

        for step, (masked_x, mlm_labels) in enumerate(dataloader):
            masked_x = masked_x.to(device)
            mlm_labels = mlm_labels.to(device)
            model.train()
            mlm_head.train()
            optimizer.zero_grad()

            # forward
            enc_out = model(masked_x) # (B, T, D)
            logits = mlm_head(enc_out) # (B, T, V)

            # Loss
            loss = loss_fn(logits.view(-1, logits.size(-1)), mlm_labels.view(-1))
```

```

        # backward
        loss.backward()
        optimizer.step()

        # metrics
        acc = mlm_accuracy(logits, mlm_labels)

        total_loss += loss.item()
        total_acc += acc

        # batch progress
        if (step + 1) % batch_size == 0:
            print(f"Epoch: {epoch+1}/{EPOCH} | Step : {step+1}/{len(dataloader)}

    avg_loss = total_loss / len(dataloader)
    avg_acc = total_acc / len(dataloader)

    print(f"\nEpoch: {epoch+1} | Avg Loss: {avg_loss:.4f} | Avg Acc: {avg_acc:.

    return train, mlm_accuracy

```

In [109...

```

%%time
train(EPOCH=5, batch_size=100)

```

Epoch: 1/5		Step : 100/1145		Loss: 9.9889		Acc: 0.0263
Epoch: 1/5		Step : 200/1145		Loss: 9.8518		Acc: 0.0423
Epoch: 1/5		Step : 300/1145		Loss: 9.5277		Acc: 0.0410
Epoch: 1/5		Step : 400/1145		Loss: 9.4870		Acc: 0.0397
Epoch: 1/5		Step : 500/1145		Loss: 9.4938		Acc: 0.0333
Epoch: 1/5		Step : 600/1145		Loss: 9.6755		Acc: 0.0342
Epoch: 1/5		Step : 700/1145		Loss: 9.8906		Acc: 0.0129
Epoch: 1/5		Step : 800/1145		Loss: 9.4334		Acc: 0.0388
Epoch: 1/5		Step : 900/1145		Loss: 9.4166		Acc: 0.0185
Epoch: 1/5		Step : 1000/1145		Loss: 9.4784		Acc: 0.0345
Epoch: 1/5		Step : 1100/1145		Loss: 9.7030		Acc: 0.0270

Epoch: 1 | Avg Loss: 9.7288 | Avg Acc: 0.0296

Epoch: 2/5		Step : 100/1145		Loss: 9.4851		Acc: 0.0339
Epoch: 2/5		Step : 200/1145		Loss: 9.8613		Acc: 0.0168
Epoch: 2/5		Step : 300/1145		Loss: 9.4639		Acc: 0.0226
Epoch: 2/5		Step : 400/1145		Loss: 9.7922		Acc: 0.0259
Epoch: 2/5		Step : 500/1145		Loss: 9.2797		Acc: 0.0072
Epoch: 2/5		Step : 600/1145		Loss: 9.7503		Acc: 0.0424
Epoch: 2/5		Step : 700/1145		Loss: 9.6094		Acc: 0.0210
Epoch: 2/5		Step : 800/1145		Loss: 9.4627		Acc: 0.0280
Epoch: 2/5		Step : 900/1145		Loss: 9.3692		Acc: 0.0093
Epoch: 2/5		Step : 1000/1145		Loss: 9.3892		Acc: 0.0275
Epoch: 2/5		Step : 1100/1145		Loss: 9.3478		Acc: 0.0500

Epoch: 2 | Avg Loss: 9.5453 | Avg Acc: 0.0310

Epoch: 3/5		Step : 100/1145		Loss: 9.3530		Acc: 0.0427
Epoch: 3/5		Step : 200/1145		Loss: 9.2701		Acc: 0.0246
Epoch: 3/5		Step : 300/1145		Loss: 9.4313		Acc: 0.0403
Epoch: 3/5		Step : 400/1145		Loss: 9.3660		Acc: 0.0367
Epoch: 3/5		Step : 500/1145		Loss: 9.5155		Acc: 0.0175
Epoch: 3/5		Step : 600/1145		Loss: 9.5493		Acc: 0.0379
Epoch: 3/5		Step : 700/1145		Loss: 9.5276		Acc: 0.0226
Epoch: 3/5		Step : 800/1145		Loss: 9.6579		Acc: 0.0168
Epoch: 3/5		Step : 900/1145		Loss: 9.8267		Acc: 0.0171
Epoch: 3/5		Step : 1000/1145		Loss: 9.6435		Acc: 0.0182
Epoch: 3/5		Step : 1100/1145		Loss: 9.7089		Acc: 0.0088

Epoch: 3 | Avg Loss: 9.4657 | Avg Acc: 0.0309

Epoch: 4/5		Step : 100/1145		Loss: 9.6457		Acc: 0.0084
Epoch: 4/5		Step : 200/1145		Loss: 9.1702		Acc: 0.0603
Epoch: 4/5		Step : 300/1145		Loss: 9.1935		Acc: 0.0336
Epoch: 4/5		Step : 400/1145		Loss: 9.4471		Acc: 0.0274
Epoch: 4/5		Step : 500/1145		Loss: 9.4813		Acc: 0.0148
Epoch: 4/5		Step : 600/1145		Loss: 9.5139		Acc: 0.0345
Epoch: 4/5		Step : 700/1145		Loss: 9.4740		Acc: 0.0286
Epoch: 4/5		Step : 800/1145		Loss: 9.1589		Acc: 0.0536
Epoch: 4/5		Step : 900/1145		Loss: 9.4925		Acc: 0.0354
Epoch: 4/5		Step : 1000/1145		Loss: 9.2130		Acc: 0.0087
Epoch: 4/5		Step : 1100/1145		Loss: 9.2830		Acc: 0.0490

Epoch: 4 | Avg Loss: 9.4287 | Avg Acc: 0.0320

```
Epoch: 5/5 | Step : 100/1145 | Loss: 9.3145 | Acc: 0.0256
Epoch: 5/5 | Step : 200/1145 | Loss: 9.2769 | Acc: 0.0431
Epoch: 5/5 | Step : 300/1145 | Loss: 9.6105 | Acc: 0.0156
Epoch: 5/5 | Step : 400/1145 | Loss: 9.4336 | Acc: 0.0603
Epoch: 5/5 | Step : 500/1145 | Loss: 9.5899 | Acc: 0.0252
Epoch: 5/5 | Step : 600/1145 | Loss: 9.7864 | Acc: 0.0317
Epoch: 5/5 | Step : 700/1145 | Loss: 9.3433 | Acc: 0.0205
Epoch: 5/5 | Step : 800/1145 | Loss: 9.5910 | Acc: 0.0561
Epoch: 5/5 | Step : 900/1145 | Loss: 9.6013 | Acc: 0.0156
Epoch: 5/5 | Step : 1000/1145 | Loss: 9.4614 | Acc: 0.0229
Epoch: 5/5 | Step : 1100/1145 | Loss: 9.3889 | Acc: 0.0488
```

```
Epoch: 5 | Avg Loss: 9.3851 | Avg Acc: 0.0338
```

```
CPU times: total: 12min 6s
```

```
Wall time: 16min 16s
```

```
Out[109...] (<function __main__.train(EPOCH, batch_size)>,
              <function __main__.mlm_accuracy(logits, labels)>)
```

Training for MY-MODEL

```
In [111...] optimizer = torch.optim.Adam(my_model.parameters(), lr=1e-5)
loss_fn = nn.CrossEntropyLoss(ignore_index=-100)
```

```
In [112...] def train2(EPOCH, batch_size):
    model.train()

    for epoch in range(EPOCH):
        total_loss = 0.0
        total_acc = 0.0

        for step, (masked_x, mlm_labels) in enumerate(dataloader):
            masked_x = masked_x.to(device)
            mlm_labels = mlm_labels.to(device)
            my_model.train()
            optimizer.zero_grad()

            # forward
            logits = my_model(masked_x)          # (B,T,D)

            # Loss
            loss = loss_fn(logits.view(-1, logits.size(-1)), mlm_labels.view(-1) )

            # backward
            loss.backward()
            optimizer.step()

            # metrics
            acc = mlm_accuracy(logits, mlm_labels)

            total_loss += loss.item()
            total_acc += acc

            # batch progress
```

```
        if (step + 1) % batch_size == 0:
            print(f"Epoch: {epoch+1}/{EPOCH} | Step : {step+1}/{len(dataloader)}

            avg_loss = total_loss / len(dataloader)
            avg_acc = total_acc / len(dataloader)

            print(f"\nEpoch: {epoch+1} | Avg Loss: {avg_loss:.4f} | Avg Acc: {avg_acc:.

        return train, mlm_accuracy
```

In [113...

```
%%time
train2(EPOCH=5, batch_size=100)
```

Epoch: 1/5		Step : 100/1145		Loss: 8.3206		Acc: 0.0513
Epoch: 1/5		Step : 200/1145		Loss: 7.3094		Acc: 0.0086
Epoch: 1/5		Step : 300/1145		Loss: 7.2164		Acc: 0.0413
Epoch: 1/5		Step : 400/1145		Loss: 7.1323		Acc: 0.0500
Epoch: 1/5		Step : 500/1145		Loss: 7.5527		Acc: 0.0667
Epoch: 1/5		Step : 600/1145		Loss: 7.8168		Acc: 0.0345
Epoch: 1/5		Step : 700/1145		Loss: 7.0425		Acc: 0.0376
Epoch: 1/5		Step : 800/1145		Loss: 7.0095		Acc: 0.0242
Epoch: 1/5		Step : 900/1145		Loss: 7.3603		Acc: 0.0413
Epoch: 1/5		Step : 1000/1145		Loss: 7.3118		Acc: 0.0377
Epoch: 1/5		Step : 1100/1145		Loss: 6.8196		Acc: 0.0320

Epoch: 1 | Avg Loss: 7.4748 | Avg Acc: 0.0316

Epoch: 2/5		Step : 100/1145		Loss: 6.5883		Acc: 0.0465
Epoch: 2/5		Step : 200/1145		Loss: 6.8491		Acc: 0.0076
Epoch: 2/5		Step : 300/1145		Loss: 6.8825		Acc: 0.0598
Epoch: 2/5		Step : 400/1145		Loss: 6.9375		Acc: 0.0484
Epoch: 2/5		Step : 500/1145		Loss: 7.1549		Acc: 0.0391
Epoch: 2/5		Step : 600/1145		Loss: 8.0582		Acc: 0.0310
Epoch: 2/5		Step : 700/1145		Loss: 6.1785		Acc: 0.0260
Epoch: 2/5		Step : 800/1145		Loss: 6.8528		Acc: 0.0348
Epoch: 2/5		Step : 900/1145		Loss: 7.0086		Acc: 0.0179
Epoch: 2/5		Step : 1000/1145		Loss: 6.6630		Acc: 0.0720
Epoch: 2/5		Step : 1100/1145		Loss: 6.8209		Acc: 0.0204

Epoch: 2 | Avg Loss: 7.0098 | Avg Acc: 0.0363

Epoch: 3/5		Step : 100/1145		Loss: 6.5084		Acc: 0.0734
Epoch: 3/5		Step : 200/1145		Loss: 6.7881		Acc: 0.0331
Epoch: 3/5		Step : 300/1145		Loss: 7.0757		Acc: 0.0342
Epoch: 3/5		Step : 400/1145		Loss: 6.7517		Acc: 0.0813
Epoch: 3/5		Step : 500/1145		Loss: 7.1729		Acc: 0.0288
Epoch: 3/5		Step : 600/1145		Loss: 7.2149		Acc: 0.0530
Epoch: 3/5		Step : 700/1145		Loss: 6.9995		Acc: 0.0479
Epoch: 3/5		Step : 800/1145		Loss: 6.7638		Acc: 0.0280
Epoch: 3/5		Step : 900/1145		Loss: 6.9435		Acc: 0.0598
Epoch: 3/5		Step : 1000/1145		Loss: 7.3219		Acc: 0.0227
Epoch: 3/5		Step : 1100/1145		Loss: 6.8243		Acc: 0.0526

Epoch: 3 | Avg Loss: 6.9023 | Avg Acc: 0.0423

Epoch: 4/5		Step : 100/1145		Loss: 6.5822		Acc: 0.0574
Epoch: 4/5		Step : 200/1145		Loss: 6.2744		Acc: 0.0720
Epoch: 4/5		Step : 300/1145		Loss: 6.5529		Acc: 0.0296
Epoch: 4/5		Step : 400/1145		Loss: 7.1019		Acc: 0.0385
Epoch: 4/5		Step : 500/1145		Loss: 6.7156		Acc: 0.0597
Epoch: 4/5		Step : 600/1145		Loss: 7.5940		Acc: 0.0522
Epoch: 4/5		Step : 700/1145		Loss: 6.7755		Acc: 0.0292
Epoch: 4/5		Step : 800/1145		Loss: 6.8159		Acc: 0.0331
Epoch: 4/5		Step : 900/1145		Loss: 6.7515		Acc: 0.0667
Epoch: 4/5		Step : 1000/1145		Loss: 6.9634		Acc: 0.0392
Epoch: 4/5		Step : 1100/1145		Loss: 6.6490		Acc: 0.0234

Epoch: 4 | Avg Loss: 6.8176 | Avg Acc: 0.0464

```
Epoch: 5/5 | Step : 100/1145 | Loss: 6.6057 | Acc: 0.0593
Epoch: 5/5 | Step : 200/1145 | Loss: 6.4663 | Acc: 0.0732
Epoch: 5/5 | Step : 300/1145 | Loss: 6.3894 | Acc: 0.1029
Epoch: 5/5 | Step : 400/1145 | Loss: 6.7908 | Acc: 0.0640
Epoch: 5/5 | Step : 500/1145 | Loss: 6.8862 | Acc: 0.0762
Epoch: 5/5 | Step : 600/1145 | Loss: 7.4306 | Acc: 0.0160
Epoch: 5/5 | Step : 700/1145 | Loss: 6.3560 | Acc: 0.0400
Epoch: 5/5 | Step : 800/1145 | Loss: 6.6936 | Acc: 0.0550
Epoch: 5/5 | Step : 900/1145 | Loss: 6.5718 | Acc: 0.0268
Epoch: 5/5 | Step : 1000/1145 | Loss: 6.6482 | Acc: 0.1038
Epoch: 5/5 | Step : 1100/1145 | Loss: 6.2557 | Acc: 0.0619
```

```
Epoch: 5 | Avg Loss: 6.7197 | Avg Acc: 0.0534
```

```
CPU times: total: 13min 49s
```

```
Wall time: 4h 50min 29s
```

```
Out[113...] (<function __main__.train(EPOCH, batch_size)>,
             <function __main__.mlm_accuracy(logits, labels)>)
```

```
In [ ]:
```

save models

```
In [114...] from pathlib import Path
root=Path('models')
root.mkdir(exist_ok=True)
path1= root / "Bert.pth"
path2= root / "mask.pth"
path3= root / "mybert.pth"
```

```
In [115...] torch.save(model,path1)
```

```
In [116...] torch.save(mlm_head,path2)
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```